

Отчет по практике

Тема: прогнозирование стоимости медицинской страховки на основе дерева решений и ансамблевых моделей.

В отчете рассматривается полный ход работы: постановка задачи, разведочный анализ данных, предобработка, построение базовой модели, борьба с переобучением, ансамблевые методы и итоговое сравнение моделей.

Структура отчета оформлена по образцу: введение, основная часть, заключение, список использованных источников и приложения.

Введение

1. Постановка задачи и описание датасета
2. Разведочный анализ и предобработка данных
3. Базовая модель и проблема переобучения
4. Ансамблевые модели: Bagging и Random Forest
5. Градиентный бустинг
6. Сравнительный анализ моделей

Заключение

Список использованных источников

Введение

Цель работы заключается в построении и сравнении моделей машинного обучения для прогнозирования стоимости медицинской страховки клиента. В качестве целевой переменной используется стоимость страховки (charges).

Практическая значимость задачи состоит в том, что страховая стоимость зависит от нескольких факторов: возраста клиента (age), индекса массы тела (bmi), количества детей (children), пола (sex), факта курения (smoker) и региона проживания (region). Анализ этих признаков позволяет не только построить прогноз, но и понять, какие факторы сильнее всего влияют на итоговую стоимость.

Для решения задачи применялись модели на основе решающих деревьев: оптимизированное дерево решений, Bagging, Random Forest и XGBoost. Качество оценивалось с помощью метрик R2, MAE и RMSE на тренировочной и тестовой выборках.

1. Постановка задачи и описание датасета

Решалась задача регрессии: по признакам клиента необходимо предсказать непрерывную величину стоимости медицинской страховки (charges). Исходный датасет содержит 1338 наблюдений и 7 столбцов.

К количественным признакам относятся возраст клиента (age), индекс массы тела (bmi) и количество детей (children). К категориальным признакам относятся пол клиента (sex), факт курения (smoker) и регион проживания (region). Целевая переменная - стоимость страховки (charges).

На первом этапе была выполнена проверка пропусков. Метод `df.isna().sum()` показал, что пропущенные значения отсутствуют, поэтому удаление строк и заполнение пропусков не потребовались.

1. Постановка задачи и описание датасета

Таблица статистик целевой переменной (charges)

Статистика	Значение для charges
Среднее	13 270.42
Дисперсия	146 652 372.15
СКО	12 110.01
Минимум	1 121.87
Максимум	63 770.43

Рисунок 1 - Статистики целевой переменной (charges)

В таблице приведены среднее, дисперсия, среднеквадратическое отклонение, минимум и максимум. Большое отличие среднего значения от максимума показывает наличие дорогих страховых случаев.

2. Разведочный анализ и предобработка данных

Для изучения целевой переменной было построено распределение стоимости страховки (charges). Распределение оказалось правосторонне асимметричным: основная масса наблюдений находится в нижнем диапазоне, но присутствует длинный правый хвост дорогих случаев.

Анализ выбросов показал, что 133 объектов, то есть 9.9% выборки, имеют стоимость страховки выше \$35 000. При детальном изучении выяснилось, что 97.7% этих объектов являются курильщиками. Это позволяет считать такие наблюдения не ошибками данных, а реальными случаями высокой стоимости страхования для курящих клиентов.

Для наглядности boxplot целевой переменной был представлен горизонтально, а распределение по признаку курения (smoker) показано отдельным boxplot. Такой вариант менее растянут по горизонтали и лучше показывает различия между курящими и некурящими клиентами.

2. Разведочный анализ и предобработка данных

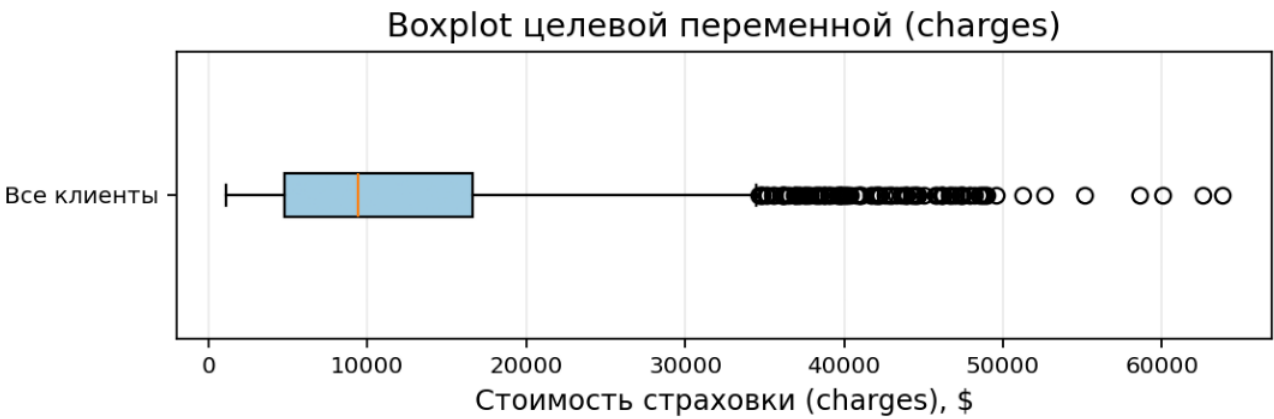


Рисунок 2 - Горизонтальный boxplot целевой переменной (charges)

Видно, что в данных есть высокие значения, расположенные далеко правее основной массы наблюдений.

2. Разведочный анализ и предобработка данных

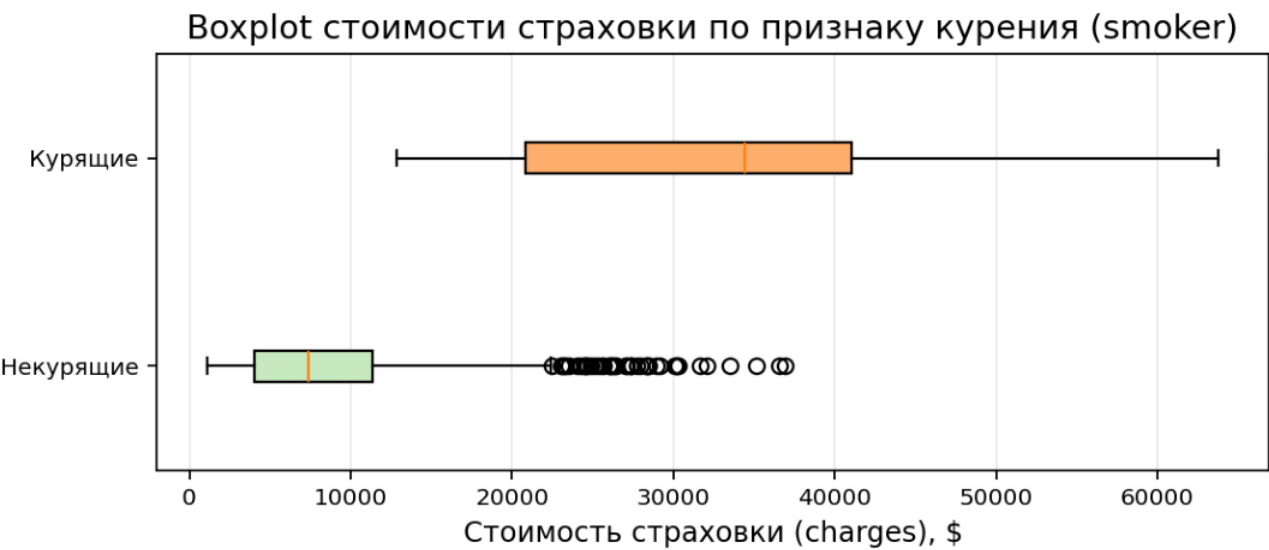


Рисунок 3 - Boxplot стоимости страховки (charges) с разделением по признаку курения (smoker)

У курящих клиентов медиана и верхние значения заметно выше, поэтому признак (smoker) является ключевым фактором стоимости.

2. Разведочный анализ и предобработка данных

Так как целевая переменная (charges) имеет сильную правостороннюю асимметрию, для обучения моделей было применено логарифмирование. Модели обучались не на исходной стоимости, а на величине $\log(\text{charges} + 1)$. Это уменьшает влияние очень больших значений и делает обучение устойчивее.

После получения прогноза выполнялось обратное преобразование. Поэтому все итоговые метрики в отчете рассчитаны не в логарифмической шкале, а в исходной шкале стоимости страховки.

$$y_{\log} = \ln(y + 1)$$

$$\hat{y} = \exp(\hat{y}_{\log}) - 1$$

```
df['charges_original'] = df['charges']  
df['charges'] = np.log1p(df['charges'])  
  
# после прогноза модели  
y_pred = np.expml(model.predict(X_test))
```

2. Разведочный анализ и предобработка данных

Сравнение корреляций для обычного и логарифмированного таргета

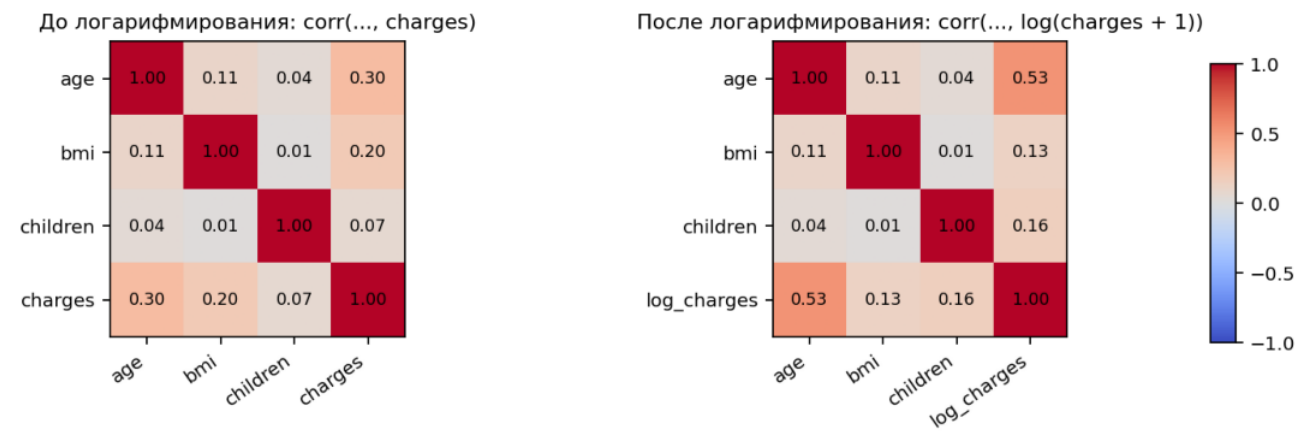


Рисунок 4 - Сравнение корреляционных матриц до и после логарифмирования целевой переменной

Сравнение показывает, как меняются парные линейные связи количественных признаков с таргетом. Для обычной стоимости страховки (`charges`) корреляция с возрастом (`age`) равна примерно 0.30, а после перехода к $\log(\text{charges} + 1)$ возраст связан с целевой переменной сильнее: около 0.53. Это показывает, что логарифмирование уменьшает влияние длинного правого хвоста и делает линейную зависимость более выраженной. Связь с индексом массы тела (`bmi`) после логарифмирования становится слабее, а связь с количеством детей (`children`) немного возрастает. Значимых сильных корреляций между самими количественными признаками не обнаружено.

2. Разведочный анализ и предобработка данных

Категориальные признаки (sex), (smoker) и (region) были преобразованы с помощью OneHotEncoder. Такой способ кодирования не создает искусственный порядок между категориями и подходит для дальнейшего обучения моделей.

С учетом перекоса выборки было принято решение использовать стратифицированное разделение данных. Для регрессии стандартная стратификация невозможна, потому что целевая переменная непрерывная. Поэтому была применена комбинированная стратификация: стоимость страховки (charges) сначала была разбита на интервалы, а затем интервалы были объединены с признаком курения (smoker). Это позволило сохранить похожее распределение стоимости и долю курильщиков в train, validation и test.

После разделения получены выборки: train - 801 объект, validation - 268 объектов, test - 268 объектов. После OneHotEncoding итоговый набор признаков содержит 8 признаков.

```
df['charges_bin'] = pd.qcut(df['charges'], q=5, duplicates='drop')
df['stratify_col'] = df['charges_bin'].astype(str) + '_' + df['smoker']

train_df, temp_df = train_test_split(
    df,
    test_size=0.4,
    random_state=42,
    stratify=df['stratify_col']
)
```

2. Разведочный анализ и предобработка данных

Фрагмент датасета после предобработки

age	bmi	children	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest	charges
60.0	29.64	0.0	1.0	0.0	0.0	0.0	0.0	9.452
56.0	33.63	0.0	1.0	1.0	1.0	0.0	0.0	10.69
23.0	27.36	1.0	1.0	0.0	1.0	0.0	0.0	7.934
18.0	34.1	0.0	1.0	0.0	0.0	1.0	0.0	7.037
28.0	26.98	2.0	1.0	0.0	0.0	0.0	0.0	8.398
53.0	26.6	0.0	0.0	0.0	1.0	0.0	0.0	9.245

Рисунок 5 - Фрагмент датасета после предобработки

В таблице видны количественные признаки, one-hot признаки категорий и логарифмированная целевая переменная (charges). Исходная шкала сохранена отдельно в поле (charges_original) и используется для расчета метрик.

Вывод по разделу 2

Таким образом, после выполненной предобработки был получен датасет без пропусков, с обработанными категориальными признаками и логарифмированной целевой переменной. Высокие значения стоимости страховки не удалялись, так как анализ показал их связь с признаком курения (smoker).

Подготовленные данные подходят для обучения моделей регрессии: целевая переменная стабилизирована логарифмированием, признаки приведены к числовому виду, а train, validation и test имеют близкую структуру благодаря комбинированной стратификации.

3. Базовая модель и проблема переобучения

В качестве базовой модели было использовано дерево решений для регрессии. Сначала рассматривалась модель без сильных ограничений, чтобы показать риск переобучения: глубокое дерево способно запоминать отдельные наблюдения обучающей выборки и хуже переносить закономерности на новые данные.

Далее была построена оптимизированная модель дерева решений. Для нее итоговая глубина дерева составила 10. На тренировочной выборке $R^2 = 0.8576$, $RMSE = 4\,622.26$, а на тестовой выборке $R^2 = 0.8399$, $RMSE = 4\,564.54$. Разница между train и test показывает, что ограничения дерева снизили переобучение, но полностью не устранили зависимость модели от структуры обучающей выборки.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$
$$MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad RMSE = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$$

3. Базовая модель и проблема переобучения

Лучшие параметры DecisionTreeRegressor

Параметр	Значение
criterion	absolute_error
max_depth	10
min_samples_split	2
min_samples_leaf	8

Рисунок 6 - Лучшие параметры оптимизированного дерева решений

В тексте используется именно термин 'оптимизированное дерево', чтобы не смешивать его с неограниченной базовой моделью.

3. Базовая модель и проблема переобучения

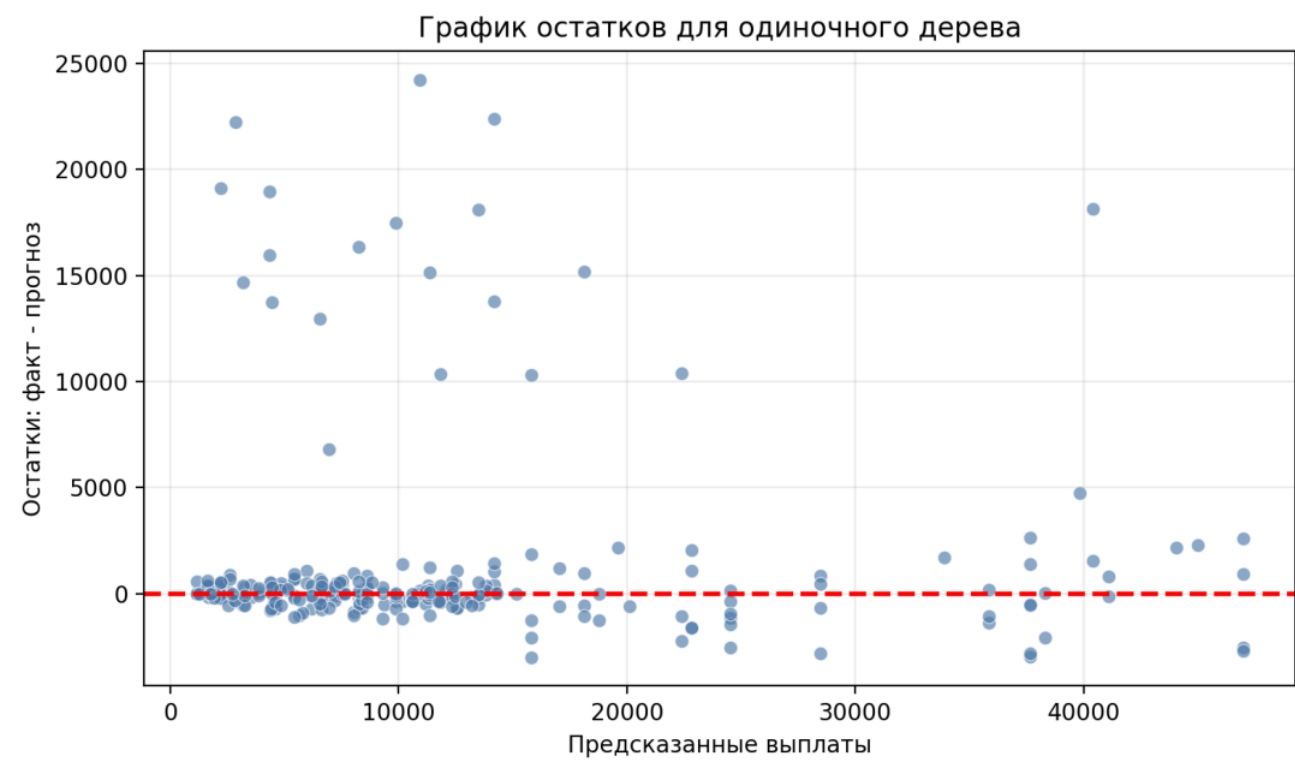


Рисунок 7 - Остатки оптимизированного дерева решений на тестовой выборке

Дискретность предсказаний связана с тем, что дерево возвращает одинаковое значение для всех объектов внутри одного листа. Крупные ошибки чаще возникают на дорогих страховых случаях.

3. Базовая модель и проблема переобучения

Для подбора гиперпараметров применялась процедура кросс-валидации. Ее смысл состоит в том, что обучающая выборка несколько раз делится на внутренние обучающие и проверочные части, а качество модели усредняется по разным разбиениям. Это снижает зависимость результата от одного случайного разделения.

Для дерева изменялись параметры: максимальная глубина (`max_depth`), минимальное число объектов для разделения вершины (`min_samples_split`), минимальное число объектов в листе (`min_samples_leaf`), критерий разбиения (`criterion`) и параметр пост-обрезки (`ccp_alpha`).

```
param_grid = {
    'max_depth': [4, 5, 6, 7, 8, 10, None],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8],
    'criterion': ['squared_error', 'absolute_error'],
    'ccp_alpha': [0.0, 0.0001, 0.001]
}

grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid=param_grid,
    cv=KFold(n_splits=5, shuffle=True, random_state=42),
    scoring=r2_original_scorer
)
```

4. Ансамблевые модели: Bagging и Random Forest

После одиночного дерева были рассмотрены ансамблевые модели. Их смысл заключается в объединении нескольких деревьев, что снижает разброс предсказаний и делает модель устойчивее.

Bagging обучает деревья на bootstrap-подвыборках и усредняет их прогнозы. Random Forest дополнительно использует случайный выбор признаков при построении разбиений, поэтому деревья внутри ансамбля становятся менее похожими друг на друга.

Для Bagging подбирались число деревьев (`n_estimators`), доля объектов (`max_samples`), доля признаков (`max_features`), а также параметры базового дерева. Для Random Forest дополнительно подбирались параметры случайного леса: (`max_depth`), (`min_samples_split`), (`min_samples_leaf`), (`max_features`), (`bootstrap`) и (`ccp_alpha`).

4. Ансамблевые модели: Bagging и Random Forest

Лучшие параметры BaggingRegressor

Параметр	Значение
n_estimators	50
max_samples	0.6
max_features	1.0
bootstrap	True
estimator__max_depth	5
estimator__min_samples_split	2
estimator__min_samples_leaf	2

Рисунок 8 - Лучшие параметры Bagging

Модель подбиралась в два этапа: сначала широкий поиск, затем уточнение вокруг найденных значений.

4. Ансамблевые модели: Bagging и Random Forest

Лучшие параметры RandomForestRegressor

Параметр	Значение
n_estimators	100
max_depth	6
min_samples_split	2
min_samples_leaf	2
max_features	0.8
bootstrap	True
ccp_alpha	0.0

Рисунок 9 - Лучшие параметры Random Forest

По сравнению с Bagging здесь важен параметр (max_features), так как случайный лес выбирает подмножество признаков при разбиениях.

4. Ансамблевые модели: OOB-score

Для моделей с bootstrap-подвыборками дополнительно применялся OOB-score. Его смысл в том, что каждое дерево обучается не на всех объектах, и оставшиеся объекты можно использовать как внутреннюю проверочную выборку без отдельного разбиения.

OOB-score сравнивался с оценкой по кросс-валидации. Близость этих значений показывает, что внутренняя оценка качества у ансамблей согласуется с KFold-кросс-валидацией. Для одиночного дерева OOB-score не применяется, потому что оно не строится на bootstrap-композиции.

```
bagging_oob = BaggingRegressor(  
    estimator=best_tree,  
    n_estimators=best_n,  
    bootstrap=True,  
    oob_score=True,  
    random_state=42  
)  
  
rf_oob = RandomForestRegressor(  
    n_estimators=best_n,  
    bootstrap=True,  
    oob_score=True,  
    random_state=42  
)
```

Вывод по разделу 4

Ансамбли улучшили качество по сравнению с оптимизированным одиночным деревом по R^2 и RMSE. При этом значения RMSE у Bagging и Random Forest оказались близкими, потому что обе модели основаны на усреднении деревьев и работают с одним и тем же ограничением данных: высокие страховые случаи остаются наиболее сложной частью выборки.

Random Forest показал небольшое преимущество за счет декорреляции деревьев через случайный выбор признаков. Однако разница с Bagging невелика, так как ключевые признаки, особенно факт курения (smoker), настолько сильны, что модели приходят к похожим разбиениям.

5. Градиентный бустинг

Градиентный бустинг строит ансамбль последовательно: каждое новое дерево исправляет ошибки предыдущих деревьев. В соответствии с планом проекта для этого этапа использовалась библиотека XGBoost, а именно модель XGBRegressor.

XGBoost является современной реализацией градиентного бустинга: кроме последовательного исправления ошибок, он использует регуляризацию, подвыборку объектов и подвыборку признаков, что помогает контролировать переобучение.

После широкого поиска лучший CV R2 составил 0.8399, после уточненного поиска - 0.8426.

Лучший шаг по validation был около 648 деревьев.

```
xgb_base = XGBRegressor(
    objective='reg:squarederror',
    random_state=42,
    n_jobs=-1,
    verbosity=0
)

xgb_search = RandomizedSearchCV(
    estimator=xgb_base,
    param_distributions=xgb_param_grid,
    n_iter=60,
    cv=KFold(n_splits=5, shuffle=True, random_state=42),
    scoring=r2_original_scorer,
    random_state=42
)
```

5. Градиентный бустинг

Лучшие параметры XGBoost

Гиперпараметр	Значение
subsample	0.9
reg_lambda	4.0
reg_alpha	0.2
n_estimators	700
min_child_weight	5
max_depth	3
learning_rate	0.01
colsample_bytree	1.0

Рисунок 10 - Лучшие параметры XGBoost после уточненного поиска

Параметры (learning_rate), (n_estimators), (max_depth), (subsample), (colsample_bytree), (reg_lambda) и (reg_alpha) управляют скоростью, сложностью и регуляризацией модели.

5. Градиентный бустинг

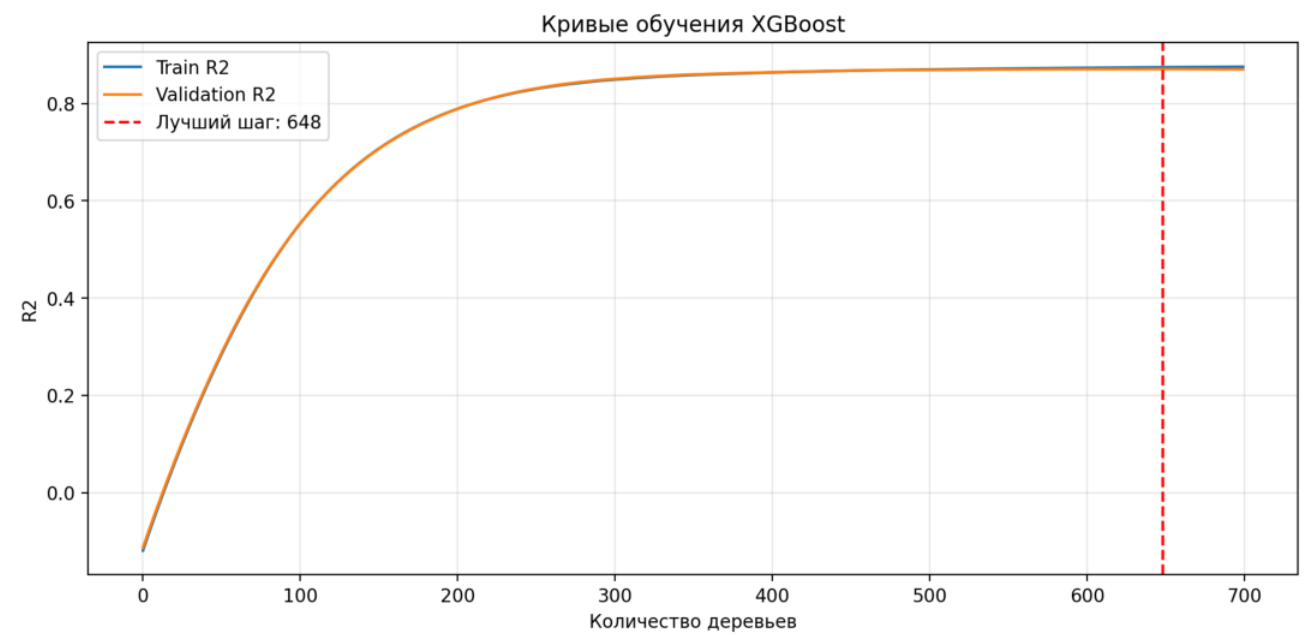


Рисунок 11 - Кривая качества XGBoost по числу деревьев

После определенного числа итераций качество на validation почти не растет, поэтому дальнейшее увеличение ансамбля может не давать практического выигрыша.

5. Градиентный бустинг

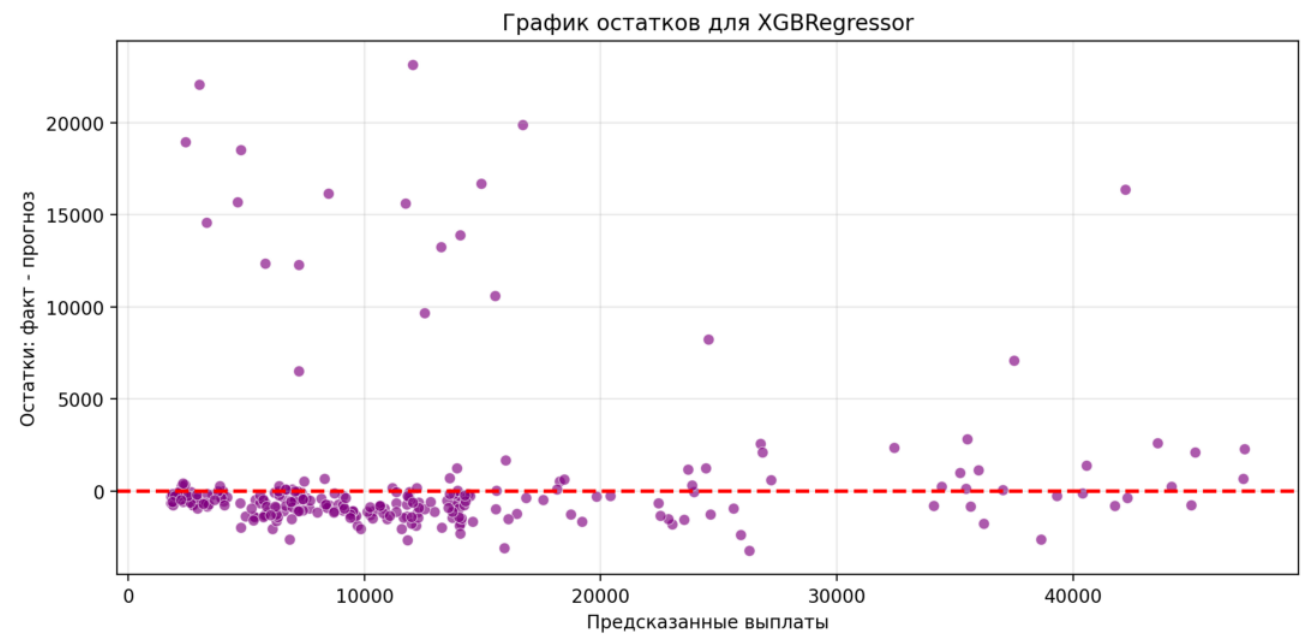


Рисунок 12 - Остатки XGBoost на тестовой выборке

Основная масса ошибок сосредоточена около нуля, но на высоких значениях (charges) сохраняется увеличенный разброс.

5. Градиентный бустинг

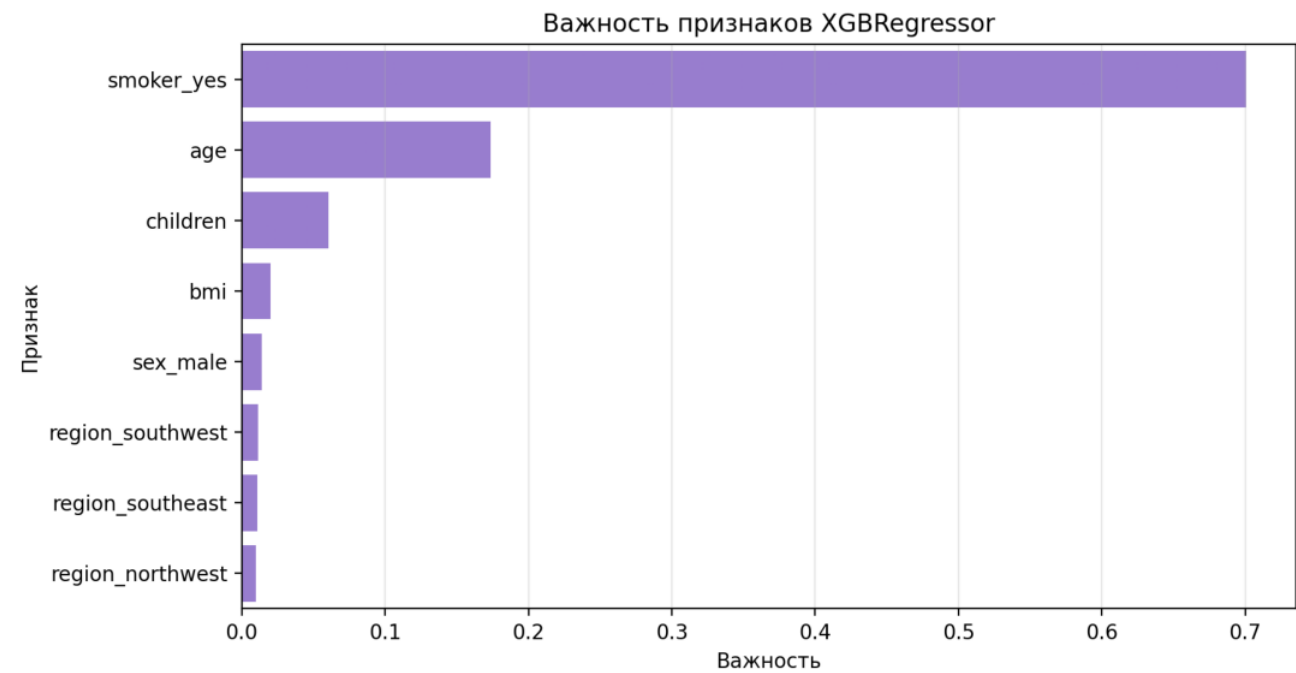


Рисунок 13 - Важность признаков XGBoost

Наиболее важным фактором является признак курения (`smoker_yes`), что подтверждает гипотезу из разведочного анализа о сильном влиянии курения на стоимость страховки.

6. Сравнительный анализ моделей

На заключительном этапе результаты всех моделей были объединены в одну таблицу. Метрики сравнивались на тренировочной и тестовой выборках, так как именно это позволяет оценить не только качество, но и степень переобучения.

Особое внимание уделялось сравнению R^2 на train и test. Если качество на train значительно выше, чем на test, модель переобучается. Ансамблевые модели показывают более стабильную разницу между train и test, чем одиночное дерево.

6. Сравнительный анализ моделей

Итоговые метрики на train и test

Модель	Выборка	R2	MAE	RMSE
Оптимизированное дерево	Train	0.8576	1 705.66	4 622.26
Оптимизированное дерево	Test	0.8399	1 723.63	4 564.54
Bagging	Train	0.8756	1 954.27	4 319.79
Bagging	Test	0.8475	2 010.78	4 454.60
Random Forest	Train	0.8855	1 838.33	4 144.22
Random Forest	Test	0.8485	1 964.76	4 440.05
XGBoost	Train	0.8733	1 962.75	4 359.46
XGBoost	Test	0.8579	1 885.83	4 300.09

Рисунок 14 - Итоговая таблица метрик на train и test для всех моделей

Метрики рассчитаны в исходной шкале стоимости страховки после обратного преобразования $\exp m1$.

6. Сравнительный анализ моделей

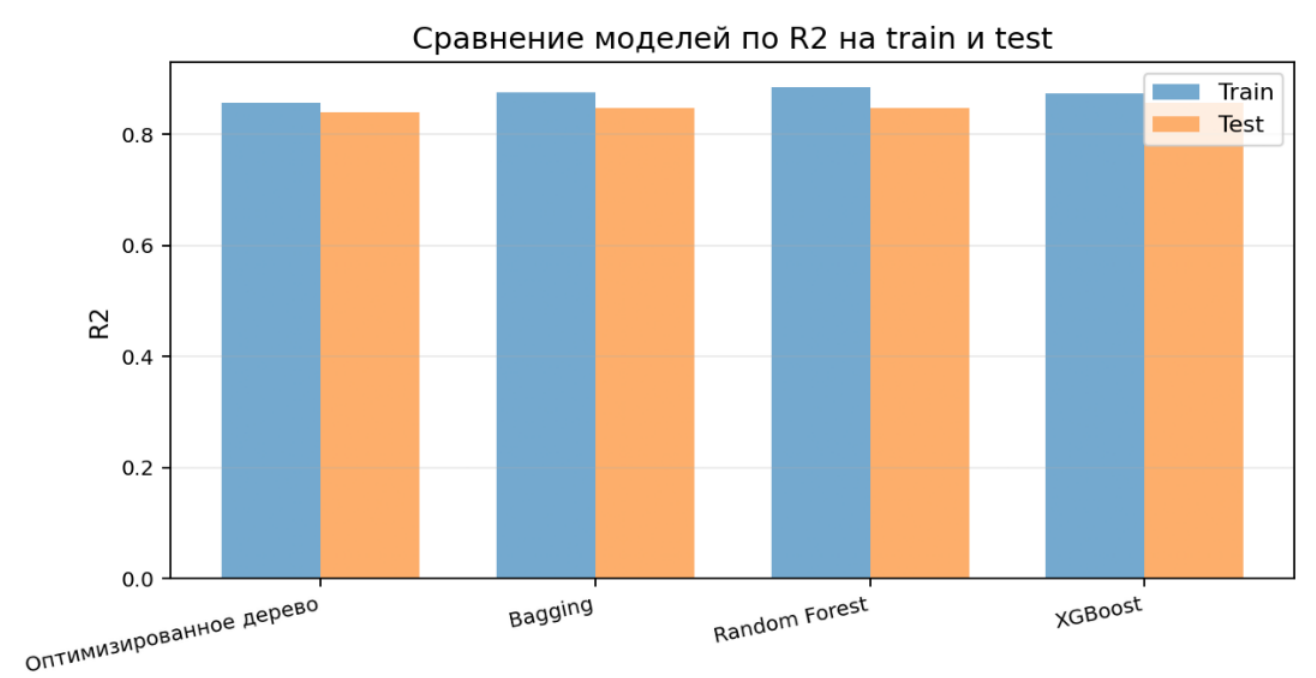


Рисунок 15 - Сравнение моделей по R2 на train и test

Столбцы разных цветов показывают, насколько качество на тестовой выборке отстает от тренировочной. Такой вид наглядно демонстрирует различия в переобучении одиночного оптимизированного дерева и ансамблей.

6. Сравнительный анализ моделей

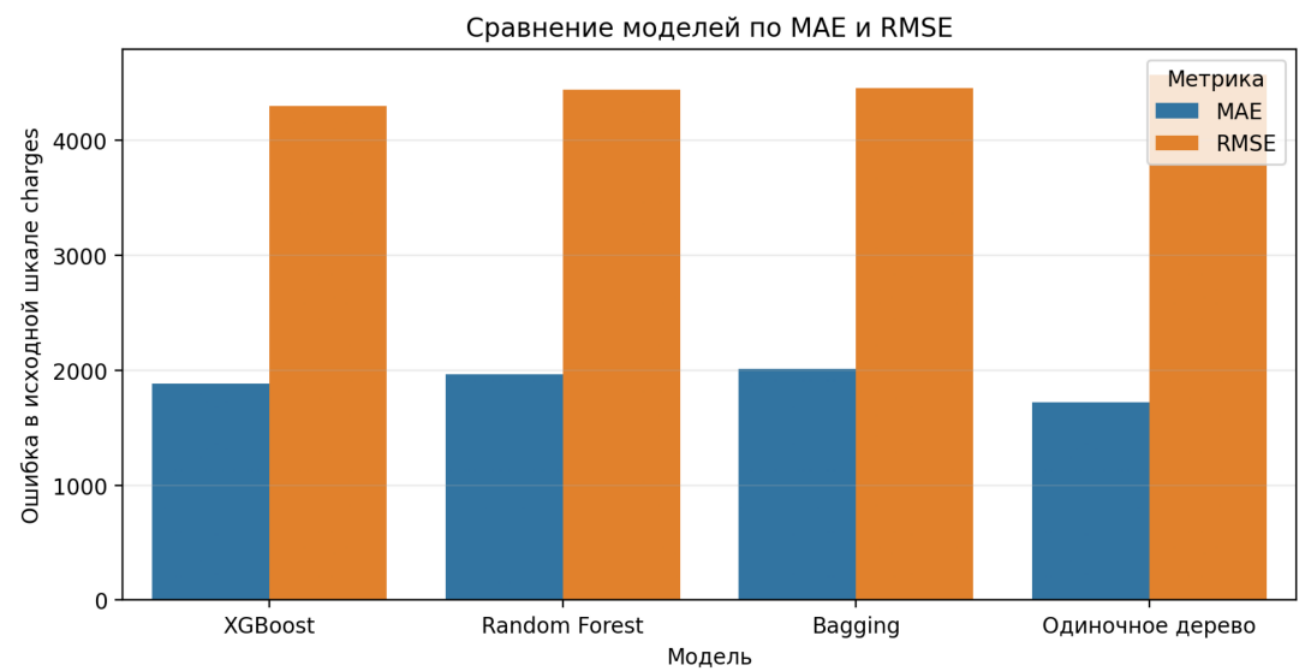


Рисунок 16 - Сравнение моделей по MAE и RMSE на тестовой выборке

RMSE у ансамблей близкие, потому что наибольший вклад в эту метрику дают редкие дорогие страховые случаи, сложные для всех моделей.

6. Сравнительный анализ моделей

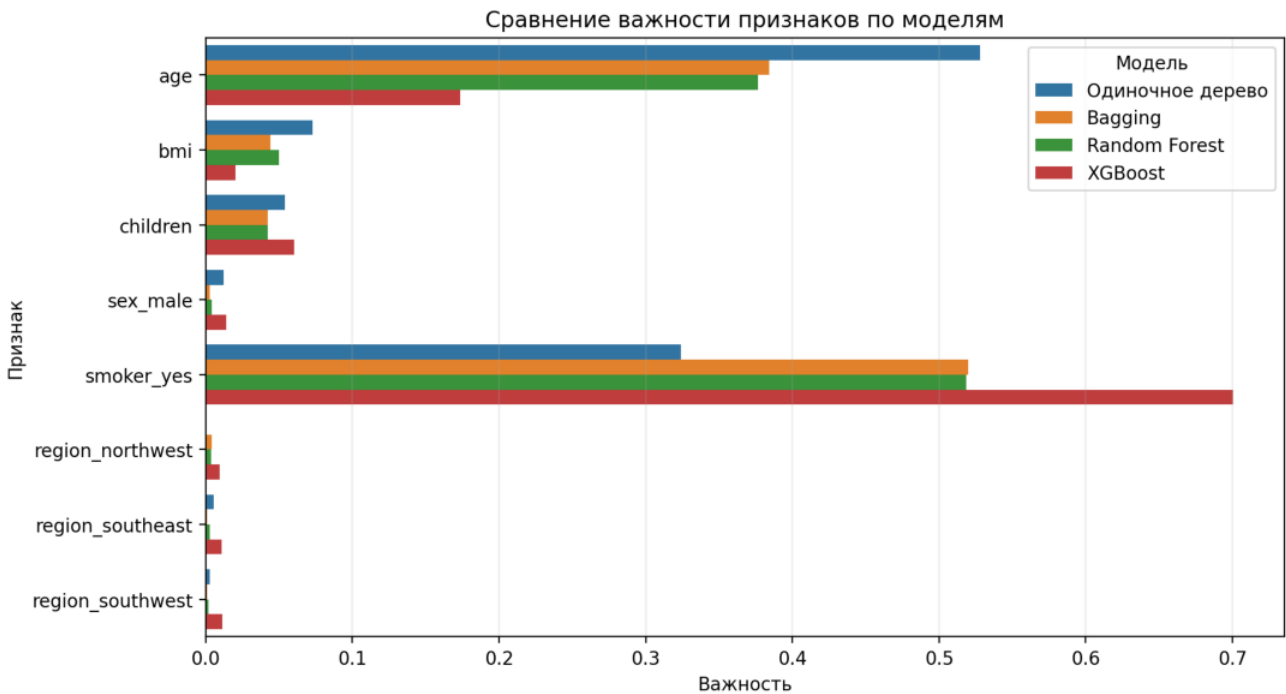


Рисунок 17 - Сравнение важности признаков

Во всех ансамблях главным фактором остается курение (smoker_yes), далее идут возраст (age) и индекс массы тела (bmi). Признаки региона и пола практически не влияют на целевую переменную.

6. Сравнительный анализ моделей

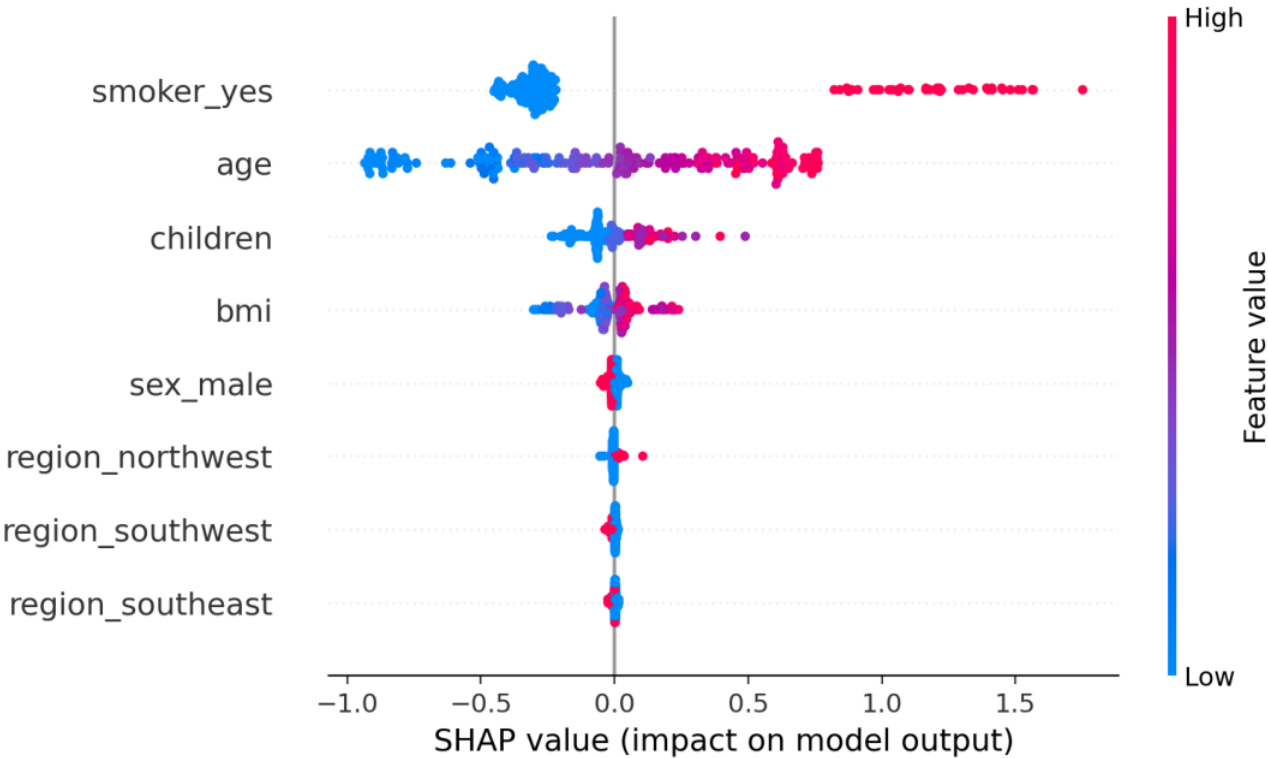


Рисунок 18 - SHAP-анализ Random Forest

SHAP-значения рассчитаны для модели, обученной на логарифмированной целевой переменной, поэтому они объясняют вклад признаков в прогноз $\log(\text{charges} + 1)$. График подтверждает высокую важность признаков (smoker_yes), (age) и (bmi).

Заключение

В ходе работы была решена задача прогнозирования стоимости медицинской страховки (charges). Были выполнены разведочный анализ, обработка категориальных признаков, логарифмирование целевой переменной, стратифицированное разделение данных и обучение нескольких моделей.

Лучшее значение R^2 на тестовой выборке показала модель XGBoost: $R^2 = 0.8579$, $MAE = 1\,885.83$, $RMSE = 4\,300.09$. XGBoost улучшил результат Random Forest по R^2 и $RMSE$, а Bagging также улучшил качество по сравнению с одиночным оптимизированным деревом по R^2 и $RMSE$.

Близкие значения $RMSE$ у ансамблей объясняются тем, что все модели сталкиваются с одной и той же сложной областью данных - дорогими страховыми случаями. Эти наблюдения в основном связаны с курением, поэтому признак (smoker_yes) стабильно оказывается главным фактором прогноза. Признаки пола и региона в сравнении с ним имеют слабое влияние.

Список использованных источников

1. Scikit-learn documentation: DecisionTreeRegressor, BaggingRegressor, RandomForestRegressor, GridSearchCV, RandomizedSearchCV.
2. XGBoost documentation: XGBRegressor and gradient boosted decision trees.
3. Pandas documentation: работа с табличными данными и расчет описательных статистик.
4. Matplotlib and Seaborn documentation: построение графиков, boxplot, heatmap и визуализация результатов.
5. SHAP documentation: интерпретация моделей машинного обучения на основе SHAP-значений.
6. Medical Cost Personal Datasets: набор данных о стоимости медицинской страховки.

Приложение

В приложении приведены ключевые формулы и фрагменты кода, которые использовались в работе. Полная реализация находится в Jupyter Notebook-файлах проекта.

$$y_{log} = \ln(y + 1)$$

$$\hat{y} = \exp(\hat{y}_{log}) - 1$$

$$RMSE = \sqrt{MSE}$$